第11回：自然言語処理との接続

NLUと音声

音声処理と認識

小野原彩香 ／ 北星学園大学

📌 この回は動画回です。Google Formsでテストを受けてください。

この回の学習目標

1. 音声処理パイプライン（ASR → NLU → DM → NLG → TTS）の全体像を説明できる
2. NLUの主要タスク（意図推定・スロット抽出・NER）を区別できる
3. SLUがNLUとどう異なるかを理解できる
4. 対話管理（DM）のアプローチを比較できる
5. LLMと音声処理の統合の最新動向を把握できる
6. PythonでWhisper・transformersを使った実装ができる

参加方法と配慮事項

この回はCBT・動画形式です。**自分のペースで**進めてください。

- 動画は一時停止・巻き戻し・字幕表示を活用してください
- CBTテストは何度でも挑戦できます（最高点を記録）
- Python実習は「読んで理解すること」でも可
- 不明点・エラーはそのままメールで質問してください

学習の進め方

[1] スライド講義を読む（または動画を視聴）
↓
[2] Python実習（Google Colab）を実行する
↓
[3] CBTテスト（5問）に挑戦する
↓
[4] 小課題を提出する（任意・加点）

音声処理パイプラインの全体像

```
音声入力
    ↓
[ASR] 音声認識（Automatic Speech Recognition）
    ↓ テキスト
[NLU] 自然言語理解（Natural Language Understanding）
    ↓ 意図・スロット・エンティティ
[DM]  対話管理（Dialogue Management）
    ↓ 行動指示・応答内容
[NLG] 自然言語生成（Natural Language Generation）
    ↓ テキスト応答
[TTS] 音声合成（Text-to-Speech）
    ↓
音声出力
```

各モジュールの役割

モジュール	入力	出力	例
ASR	音声波形	テキスト	「東京から大阪まで新幹線を予約して」
NLU	テキスト	意図・スロット	意図：鉄道予約, 出発地：東京
DM	意図・スロット・履歴	行動計画	日時が未確定 → 日時を質問
NLG	行動計画	テキスト	「出発日時をお教えください」
TTS	テキスト	音声	合成音声で読み上げ

NLU タスク①：意図推定（Intent Classification）

ユーザーが「何をしたいか」を分類するタスク。

```
入力：「リビングの電気を消して」
出力：Intent = TurnOff_Light, Room = リビング

入力：「今日の天気は？」
出力：Intent = GetWeather, Date = 今日
```

アプローチ	手法	特徴
ルールベース	正規表現・キーワードマッチ	高速・解釈可能
機械学習	SVM, ロジスティック回帰	特徴量エンジニアリングが必要
深層学習	BERT, RoBERTa等	文脈を理解、高精度

NLU タスク②：スロット抽出（Slot Filling）

意図を実現するために必要な情報を抽出するタスク。

```
# 飛行機予約の例
入力：「来週の金曜日に東京から福岡まで飛行機を予約したい」

スロット抽出結果：
{
  "departure_city": "東京",
  "destination_city": "福岡",
  "travel_date": "来週の金曜日",
  "transport_mode": "飛行機"
}
# 未充填スロット → システムが追加で質問する
```

NLU タスク③：固有表現認識（NER）

テキスト中の固有名詞を種別にタグ付けするタスク。

> 入力：「山田太郎さんが2024年3月に東京大学で発表した論文を探して」
>
> NER結果：
> [山田太郎]PER が [2024年3月]DATE に [東京大学]ORG で発表した論文を探して

タイプ	説明	例
PER	人名	山田太郎
LOC	地名	東京、北海道
ORG	組織名	トヨタ、文部科学省
DATE/TIME	日時表現	3月15日、来週月曜

SLU — 話し言葉特有の課題

NLU と SLU（Spoken Language Understanding）の主な違い：

観点	NLU	SLU
入力	書き言葉テキスト	話し言葉テキスト（ASR出力）
句読点	あり	なし
フィラー	なし	あり（「えー」「あの」等）
誤り	テキスト入力ミス	ASR誤認識（同音異義語等）

書き言葉：「明日の会議室を予約したい」
話し言葉：「えーと、あしたの、あのー、かいぎしつ、をーっと、よやくしたいんですけど」

対話管理（Dialogue Management）

DM は会話の「流れ」をコントロールするモジュール。

```
# ルールベースDMの例
if intent == "鉄道予約":
    if departure_city == None:
        action = "出発地を質問する"
    elif destination_city == None:
        action = "目的地を質問する"
    elif travel_date == None:
        action = "日時を質問する"
    else:
        action = "予約APIを呼び出す"
```

利点：動作が予測可能、デバッグ容易
欠点：ルールが増えると管理困難、想定外の入力に弱い

対話状態トラッキング（DST）

```
ターン1: ユーザー「東京から大阪に行きたい」
  State : {departure: 東京, destination: 大阪, date: ?, count: ?}

ターン2: ユーザー「来週の土曜」
  State : {departure: 東京, destination: 大阪, date: 来週土曜, count: ?}

ターン3: ユーザー「2人で」
  State : {departure: 東京, destination: 大阪, date: 来週土曜, count: 2}
  → 全スロット充填 → 予約処理へ
```

LLM との接続 — アーキテクチャの変化

従来のパイプライン型

```
音声 → ASR → テキスト → NLU → DM → NLG → TTS → 音声
```

LLM統合型

```
音声 → ASR → テキスト → LLM（GPT-4等） → テキスト → TTS → 音声
                          ↑
                  NLU・DM・NLGを統合
```

LLMは Few-shot プロンプトで多様な意図を処理し、
プロンプトに対話履歴を渡すことで文脈も保持する。

GPT-4o の音声モード（2024〜）

従来の音声 → テキスト変換を経ずに、**直接音声から音声へ**処理。

```
従来：音声 → テキスト変換 → LLM → テキスト → 音声合成
GPT-4o：音声 → [GPT-4o（マルチモーダル）] → 音声
```

革新的な点

- 韻律・感情を音声から直接認識・表現
- テキスト変換による情報ロスがない
- 平均応答遅延：約320ミリ秒（従来の半分以下）
- 笑い・息遣い等のパラ言語情報の認識と生成

LLM統合のトレードオフ

観点	パイプライン型	LLM統合型
精度制御	各モジュールで個別チューニング可	LLMに依存
解釈可能性	各ステップを確認可能	ブラックボックス化
コスト	比較的低	API利用料が高い
柔軟性	ドメイン外の発話に弱い	幅広い対話に対応
プライバシー	オンデバイス化しやすい	クラウド依存が多い

Python実習①：SpeechRecognition + 意図分類

```python
import speech_recognition as sr
import json

# 意図とキーワードの対応表（ルールベース）
INTENT_PATTERNS = {
    "天気確認": ["天気", "気温", "雨", "晴れ"],
    "アラーム設定": ["アラーム", "目覚まし", "タイマー"],
    "電気操作": ["電気", "ライト", "照明", "消して", "つけて"],
}

def classify_intent(text: str) -> dict:
    intent_scores = {}
    for intent, keywords in INTENT_PATTERNS.items():
        score = sum(1 for kw in keywords if kw in text)
        if score > 0:
            intent_scores[intent] = score / len(keywords)
    best = max(intent_scores, key=intent_scores.get) if intent_scores else "不明"
    return {"text": text, "intent": best}
```

Python実習②：Whisper + 感情分析

```python
import whisper
from transformers import pipeline

# モデルの読み込み
whisper_model = whisper.load_model("small")
sentiment_analyzer = pipeline(
    "sentiment-analysis",
    model="lxyuan/distilbert-base-multilingual-cased-sentiments-student"
)

def speech_to_sentiment(audio_file: str) -> dict:
    # ステップ1: 音声認識（Whisper）
    transcription = whisper_model.transcribe(audio_file, language="ja")
    transcript = transcription["text"]

    # ステップ2: 感情分析（transformers）
    sentiment = sentiment_analyzer(transcript[:512])[0]
    return {"transcript": transcript, "sentiment": sentiment["label"],
            "score": sentiment["score"]}
```

評価指標まとめ

指標	対象	説明
WER	ASR	単語誤り率。低いほど精度高
Intent Accuracy	NLU	意図分類の正解率
Slot F1	NLU	スロット抽出のF1スコア
BLEU / ROUGE	NLG	n-gram一致率
MOS	TTS	人間による音声品質評価（1〜5点）
Task Completion Rate	対話全体	タスク完了できた割合

まとめ：第11回

- **音声対話パイプライン**：ASR → NLU → DM → NLG → TTS の直列構成
- **NLUの3大タスク**：意図推定・スロット抽出・固有表現認識
- **SLU**：フィラー・認識誤りを含む話し言葉に対応したNLU
- **対話管理**：ルールベースと機械学習の二大アプローチ
- **LLM統合**：NLU・DM・NLGをまとめて置き換えつつある

CBTテスト・次回予告

> **📌 CBTテスト（Google Forms）**
>
> 5問・何度でも挑戦可・最高点を記録します。
> 締め切り：翌週末 23:59

第12回（オンライン同期）：音声処理と認知科学・言語学

- 言語発達と音声知覚（臨界期仮説・音韻発達）
- 神経多様性と音声処理（ディスレクシア・APD・ASD）
- 人間と機械の音声認識の比較
- ディスカッション：「音声インタフェースはバリアかツールか？」